

CV-lab4实验报告

杨子涵

2025 年 11 月 27 日

1 函数实现说明

1.1 task1_compute_disparity_map_simple函数

输入是两张从不同角度拍摄的图像，窗口大小，视差范围，评估函数，输出是以图像1为基准的disparity_map。实现思路是，当我们要计算视差图上(u,v)位置的的值的时候，就比较第一张图中以(u,v)为中心的窗口和第二张图中以(u-d,v)为中心的窗口的相似程度，SAD是记录像素误差的绝对值之和，SSD是记录像素误差的平方之和，normalized_correlation是记录两张图像素的相关系数。然后对d在disparity_range的范围中遍历取最优就行了。注意SAD和SSD我们希望结果越小越好，而normalized_correlation我们希望结果越大越好。

代码中注意先将图像转化为float类型再计算，因为uint8类型只能表示正数，小减大会溢出成一个很大的正数。如果没有事先转化为float类型的话，SAD函数会生成错误的结果。（如图1所示）

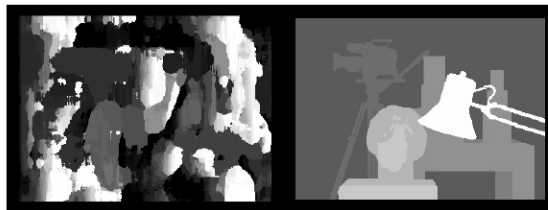


图 1: 没有把uint8类型转换为float类型的话，SAD的错误结果

我尝试了 `window_sizes = [3, 9, 15, 21, 27, 33, 39]`, `disparity_ranges = [(0, 8), (0, 16), (0, 24), (0, 32)]`, `matching_functions = ['SSD', 'SAD', 'NC']`,

生成出了结果并且记录了运行时间。

有趣的是，window_size=3,9,15时运行时间基本随 disparity_range 增大而增大，但 window_size=21 时运行时间随 disparity_range 先增大后减小（如图2a所示）；disparity_range 为 (0,8) 和 (0,16) 时，运行时间随 window_size 增大变化不大，而 disparity_range 为 (0,24) 和 (0,32) 的时候，在 window_size=21 时运行时间突然减少（如图2b所示）！把所有结果汇总到一个图里，可视化效果如图3所示。

这可能是由于代码中都是 numpy 的向量化操作，所以 window_size 变化时运行时间变化不明显；而 disparity_range 变化时，循环的次数变多了，所以运行时间会对应增加一些。

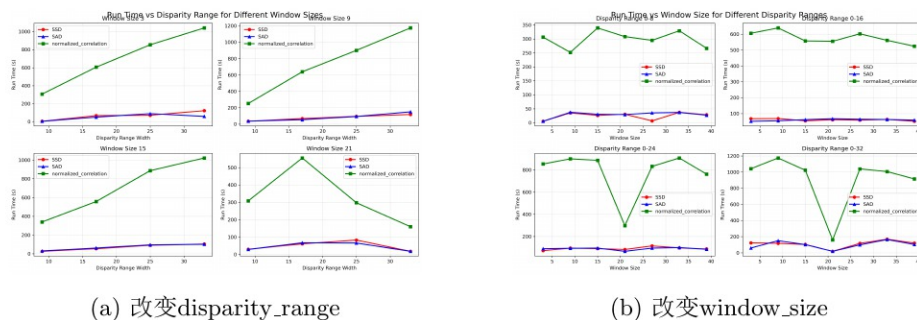


图 2: 不同参数下的运行时间变化

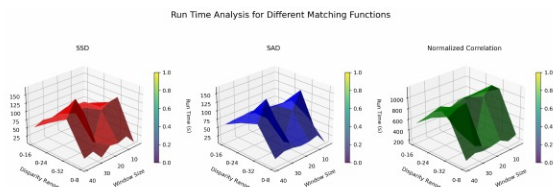
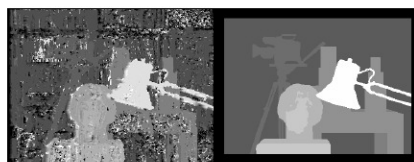


图 3: 统一的可视化图表

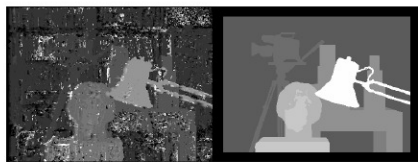
下面展示了固定window_size=3，改变disparity_range和matching_function的可视化结果（图4、5、6）。



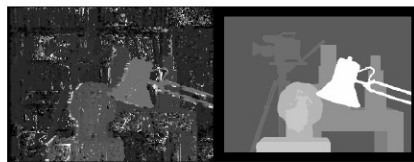
(a) disparity_range=(0,8)



(b) disparity_range=(0,16)

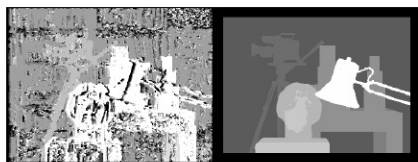


(c) disparity_range=(0,24)

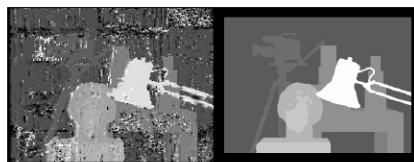


(d) disparity_range=(0,32)

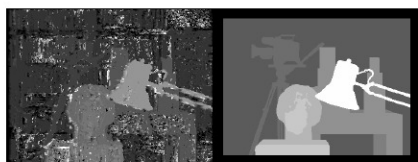
图 4: matching_function='SAD',window_size=3



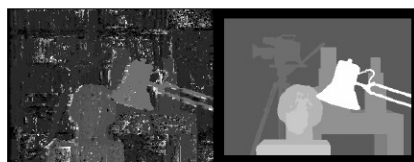
(a) disparity_range=(0,8)



(b) disparity_range=(0,16)



(c) disparity_range=(0,24)



(d) disparity_range=(0,32)

图 5: matching_function='SSD',window_size=3

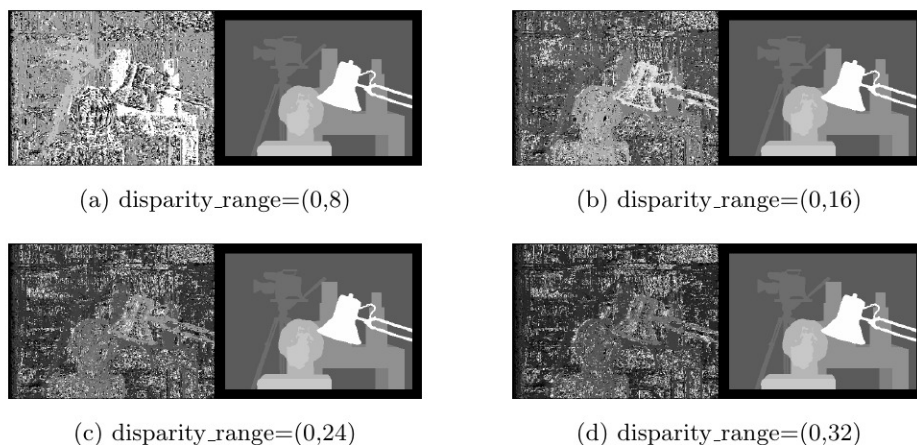


图 6: matching_function='NC', window_size=3

可以看出，当 $\text{disparity_range} = (0, 8)$ 时三种函数的效果都不好；当 $\text{disparity_range} = (0, 16), (0, 24), (0, 32)$ 时 SAD 和 SSD 的效果类似且都较好，而 normalized_correlation 的效果图中仍然有大量噪点。

值得注意的是， $\text{disparity_range} = (0, 16), (0, 24), (0, 32)$ 时总体效果类似，但是 disparity_range 越大时图片整体越暗。这是因为生成视差图的最后一步需要把视差归一化到 $[0, 1]$ 之间（具体的说，就是 `np.maximum(disparity_map, 0.0) / (disparity_map.max() + 1e-10)` 这一行代码）。当 disparity_range 很大时就有可能找到视差很大的错误匹配，这样在归一化时会导致分母很大，从而使最终归一化后的 disparity 很小，因此在图里很暗。

另外，在 $\text{disparity_range} = (0, 8)$ 时可以清晰地看到摄像机，而 $\text{disparity_range} = (0, 16), (0, 24), (0, 32)$ 时则几乎看不到，这是因为摄像机位置的视差很小（可参考 `gt`，摄像机与背景的颜色差别不大），当 disparity_range 很大时归一化的分母就很大，因此摄像机处的视差就很小，因此几乎融合在了背景中。

最终我选择的最优 disparity_range 是 $(0, 24)$ ，因为它可以较好地兼顾运行时间和效果。

下面展示了固定 $\text{disparity_range}=(0, 24)$ ，改变 window_size 和 matching_function 的可视化结果（图7、8、9）。

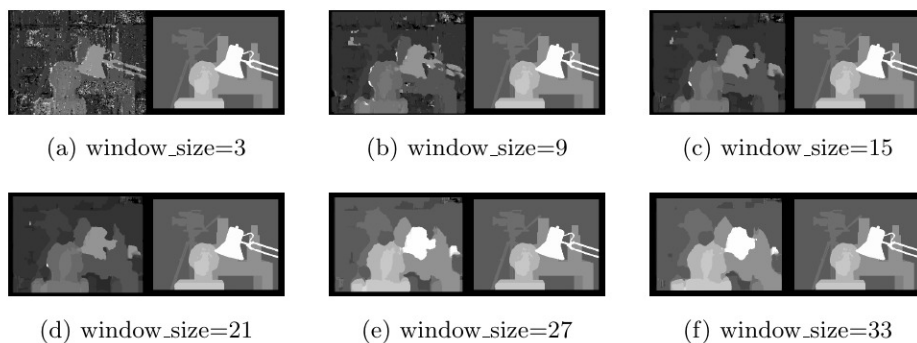


图 7: matching_function='SAD', disparity_range=(0,24)

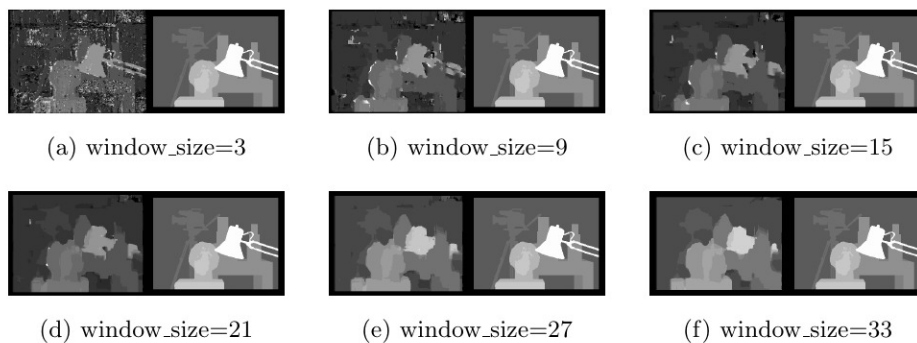


图 8: matching_function='SSD', disparity_range=(0,24)

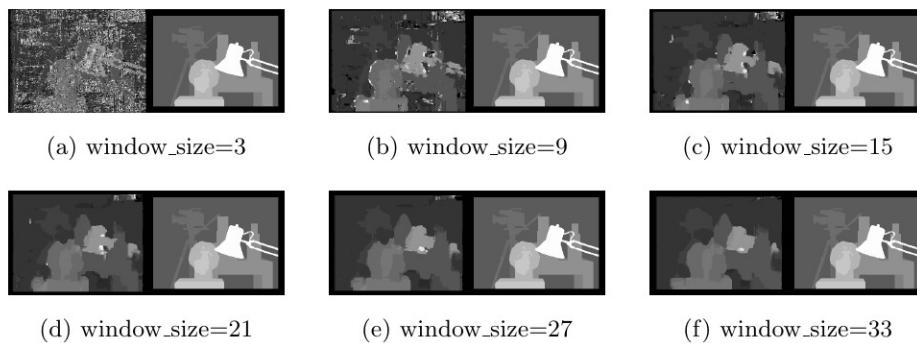


图 9: matching_function='NC', disparity_range=(0,24)

可以看出，window_size较小时图像噪点较多，不过高频细节被保留得很好（比如台灯很细的两根平行的杆子）；window_size较大时图像噪点完全

被消除(因为大的window就相当于是一个filter), 变得很平滑, 但是高频细节丢失(比如台灯的两根杆子), 并且视差图中的物体出现一定变形(比如头像雕塑的轮廓有变形)。这其实是要细节和要平滑的trade off, 很难说窗口大还是小的时候效果更好。

这里我选择的最优window_size是3, 因为这样生成的视差图里细节较多。

1.2 Problem Set 4 Questions

1.2.1 How does the running time depend on window size, disparity range, and matching function?

window_size增大, 运行时间变化不大。disparity_range增大, 运行时间一般增大。SAD与SSD的运行时间差不多, 且都明显小于normalized_correlation。(具体分析如前)

1.2.2 Which window size works the best for different matching functions?

对所有matching_function, window_size为3时总体效果最好。(window_size为3时细节最多, window_size不小于15时视差图就很平滑。具体分析如上)

1.2.3 What is the maximum disparity range that makes sense for the given stereo pair?

固定 window_size = 3, matching_function = SAD。不断增大 disparity_range, 可以看到图片越来越暗, 物体轮廓变形越来越大。比如图10是 disparity_range = (0, 64) 和 (0, 128) 时候的结果。综合来看, 有意义的 disparity_range 大概要满足 $\max_disparity \leq 64$ 。

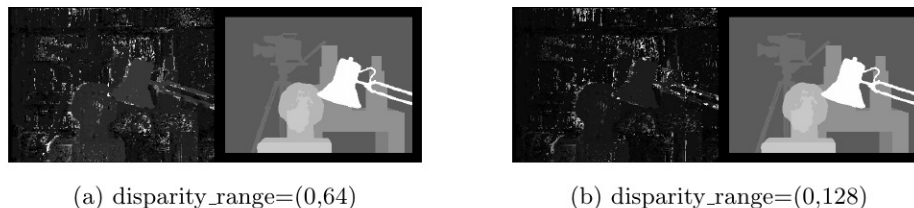


图 10: 大视差范围下的效果对比

1.2.4 Which matching function may work better for the given stereo pair?

SAD效果最好。(SSD与SAD时间和效果差别不太大; normalized_correlation时间明显比SAD、SSD长, 且效果与SAD、SSD差不多)

1.2.5 Discuss the trade-offs between different hyperparameters on quality and time.

如果需要运行时间较短, 那就需要选择较小的disparity_range, 并且选择SAD或SSD, window_size最好选小一点的。

如果需要生成视差图保留较多细节, 则需要选择较小的window_size, 较大的disparity_range, 并且配合SAD或SSD。

如果需要生成的视差图较平滑(去除噪点), 则需要选择较大的window_size。此时disparity_range和matching_function不关键。

1.2.6 Choose the best hyperparameters and show the corresponding disparity map.

我选择的是window_size=3, disparity_range=(0,24), matching_function=SAD。可视化效果如下:

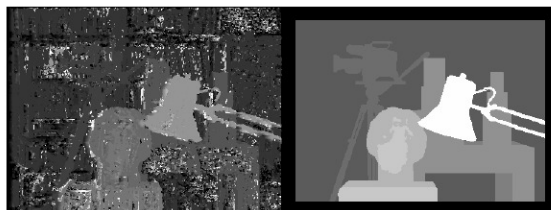


图 11: 最佳参数下的视差图: SAD, window_size=3, disparity_range=(0,24)

1.2.7 Compare the best disparity map with the ground truth map, discuss the differences and limitations of basic stereo matching.

我生成出的最好的disparity_map和ground truth的在物体轮廓上差别不大, 但是主要有如下局限:

(1) 在纹理不太明显（或者有重复）的地方（如背景书架上并排摆放的同样的书）算法容易产生歧义，于是生成的disparity_map会有噪点。

(2) 遮挡区域无法正确计算视差。（也是和gt有区别的重要原因）

改进方法就是引入non-local的算法。因为task1就是在一个范围内求局部最优，没有考虑到相邻的像素的视差大概率相近、在一张图片里更靠左的像素在另一张图片里也会更靠左等问题。同时，引入插值等修正方法可以估计出被遮挡区域的视差，使得视差图效果更好。

1.3 task2_compute_depth_map和task2_visualize_pointcloud函数

核心公式就是 $depth = \frac{focal_length \times baseline}{disparity}$ 。这里我选择的 $baseline = 100$, $focal_length = 100$ 。生成的可视化效果如下¹²：

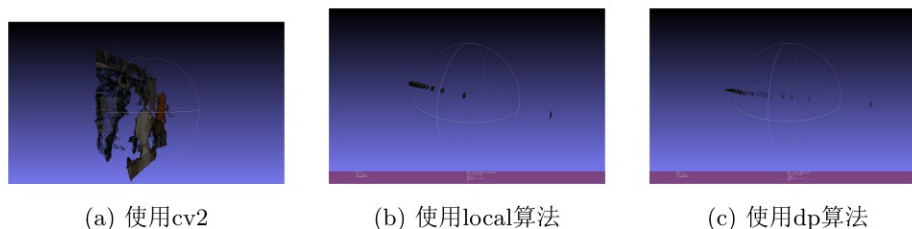


图 12: 不同算法生成的点云对比（naive实现）

可以看到，使用cv2得到的视差图生成的点云效果不错，接近example；但是使用local算法和dp算法得到的视差图生成的点云效果很奇怪。询问AI可知cv2的StereoBM函数在计算视差的时候会乘以16！

AI的解释是，StereoBM内部实际上使用16倍放大的整数值来存储视差，所以在使用的時候需要将视差除以16。StereoBM使用16倍缩放是为了：

1. 在保持整数运算效率的同时获得子像素精度
2. 平衡计算速度和精度需求
3. 优化内存使用

由此可知，为了让三种视差图在同样的超参数baseline、focal_length下效果接近，应该将我们生成disparity_map的算法中最后的结果都乘以16。将结果乘以16之后的点云如下¹³：

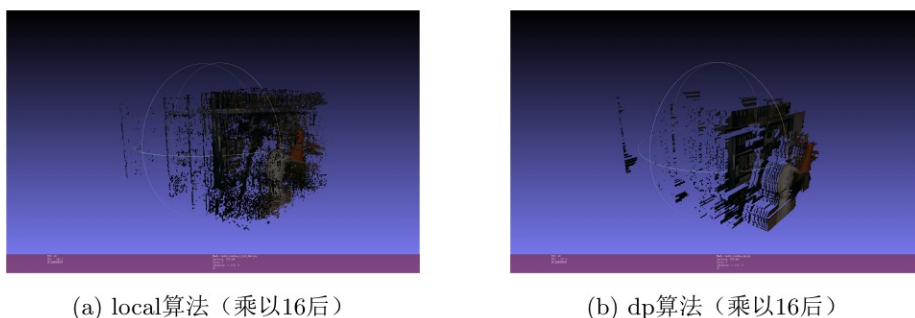


图 13: 乘以16后的点云效果

可以看到，深度的数量级已经对上了。local算法生成的视差图对应的点云里的点比较不连续，这是因为生成的视差图里本身就有很多噪点。

而用dp算法生成的视差图得到的点云已经和cv2的、example的比较相似了，而且点云中的点也较为连续，美中不足是其中有一些深度很大的点，这可能是由于它们在图像边缘，无法通过插值修复误差。

我们改进的方法是在用视差图生成点云时，排除掉深度过大的点。经过一些调参，过滤掉深度大于160的点，即可得到如下点云（图14），可以看到dp生成的点云和example已经相当相似了！

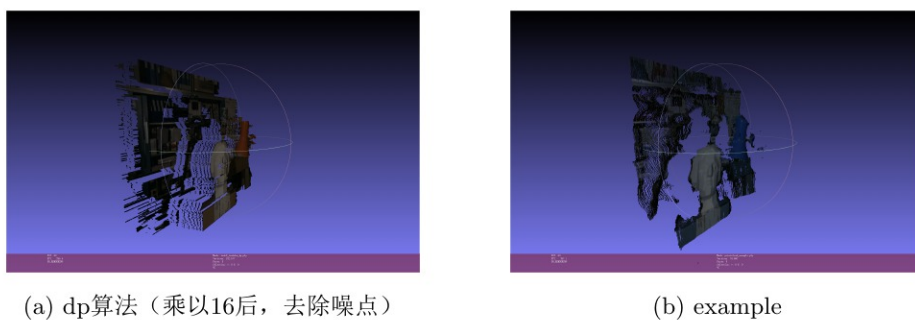


图 14: 乘以16后的点云效果

1.4 task3_compute_disparity_map_dp函数

输入是两张从不同角度拍摄的图像，输出是以图像1为基准的disparity_map。实现思路是对于每一行像素，用动态规划的方法来求解匹配。具体地说， $C[i,j]$ 这个二维数组表示图像1的前i个像素和图像2的前j个像素匹配上时的总

代价。它由之前状态计算出来的方式是：如果图像1的第*i*个像素和图像2的第*j*个像素匹配，那就再加上 $C[i-1,j-1]$ ；如果不匹配，那可能是图像1（图像2）像素被遮挡，这时候就要在 $C[i-1,j]$ （ $C[i,j-1]$ ）的基础上加上遮挡的惩罚Oclusion。代码中的 $C[i,j]$ 实际对应的是第一幅图的下标为*i*-1的像素和第二幅图的下标为*j*-1的像素，这是因为把这两个下标都整体往后推一位之后就不用单独初始化边界条件了。 $B[i,j]$ 这个二维数组用于记录当前状态是由哪一个状态转移来的，这样最后就能一步步推出完整的路径。代码中需要注意 $\min1=C[i-1,j-1]+np.abs(ref_img[row,i-1]-sec_img[row,j-1])$ 这里直接减会发生溢出，因为图像的类型是uint8，相减会得出负数。所以要先转化成int。这样实现出来的效果如图15所示，可以看到有很多延伸出来的细线。这是因为动态规划是对每一行分别求解，所以行与行之间可能不连续。



图 15: 朴素的实现,运行时间约3分钟

代码中可以调节的超参数有occlusion(在动态规划转移过程中没有匹配上的惩罚值)和max_disparity(最大的视差，代表搜索范围)。

下面我们尝试不同的优化：

1.在dp转移的时候不只考虑两个像素之间的差距作为代价，而是再加上这一行的disparity和上一行的disparity之间的差作为代价。这样的好处是鼓励与上一行之间差距更小的解。应用这个优化之后的效果如图16所示，可以看到不连续的细线消失了很多。

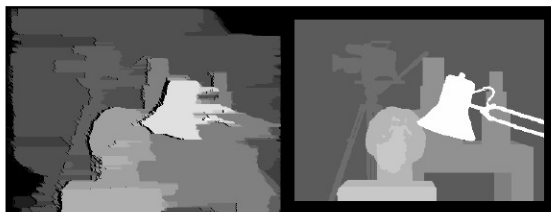


图 16: 考虑了相邻行的连续性,运行时间约8分钟

2.加入左右一致性检查。设 d 是已经求出的disparity, 我们检查图像1的 (y,x) 位置处的像素与图像2的 $(y,x-d)$ 位置处的像素是否一样。对于不一样的那些点, 可能是occlusion或者纯粹的错配, 需要修正他们的disparity。具体的方法就是利用正确匹配的那些点的disparity, 通过线性插值计算出错配的位置的disparity。可视化效果如图17所示, 可以看到, 由于occlusion导致的物体周围的黑色轮廓完全消失了。



图 17: 加入了左右一致性检查,运行时间约10分钟

但是可视化效果仍有很多不足, 比如相机的支架和台灯的支架处的细节没有被很好地还原出来。这可能是由于细节处gt视差本身变化就比较大, 但是我们的动态规划算法里会惩罚这种剧烈变化。

我写了一个代码用于比较threshold=[5, 10, 15, 20, 25, 30, 40, 50, 75, 100]时效果差异并生成了可视化图片(图19)。每张图的六张图片分别是:

第1行第1列: 原始视差图 (插值前动态规划的匹配结果)。可以看到原始的匹配质量, 包含很多空洞和噪声。

第1行第2列: 有效点分布图, 其中白色的是通过一致性检查的点 (即有效点), 黑色的是无效点。可以看到threshold=5时无效点几乎布满了各个物体的轮廓。而threshold增大时有效点越来越密集, 不过变密集的速度逐渐减缓, 这也就是说, 大部分的点在threshold比较小的时候就被过滤掉了。具体地说, 有效点的比例随threshold变化的可视化图表如下 (图18)。可以看到threshold大于25时valid_ratio基本上就不再增长了, 所以threshold=25是不错的选择。

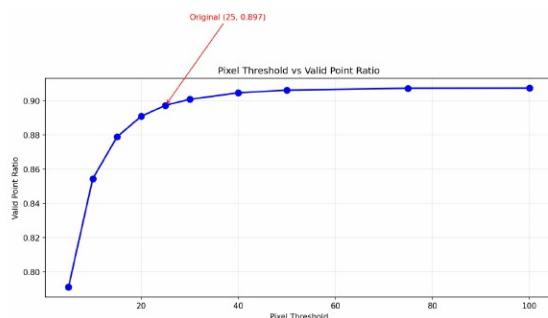


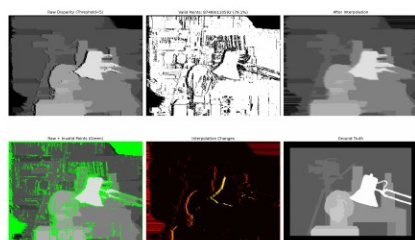
图 18: 有效点比例随threshold变化曲线

第1行第3列: 插值后的视差图, 即经过线性插值填充后的最终结果, 可以看到不同threshold下的效果几乎一样, 这说明插值的方法相当好地提高了无效点处disparity的质量。

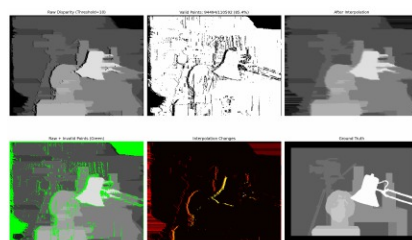
第2行第1列: 标记处了无效点的视差图。这样可以很清晰地看出, 有少量无效点游离在大片有效点中, 可能是由于图像本身的噪声造成的; 还有大量无效点在物体的轮廓线上, 在这里gt的视差剧烈变化, 所以容易产生错配。

第2行第2列: 插值前后差异的热力图 (红色越深表示插值改变越大)。可以看到物体轮廓处, 和视差图边缘处, 插值改变很大。

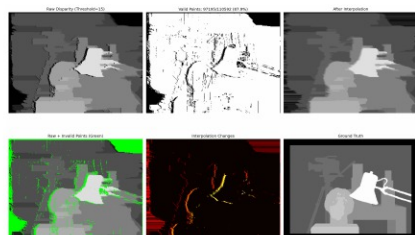
第2行第3列: ground truth视差图, 用作参考。



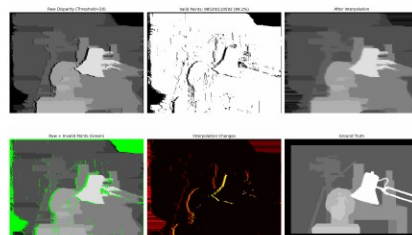
(a) threshold=5



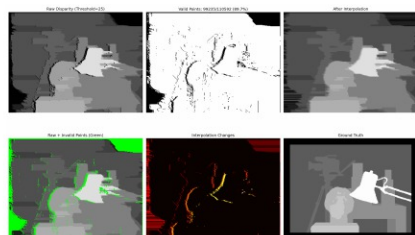
(b) threshold=10



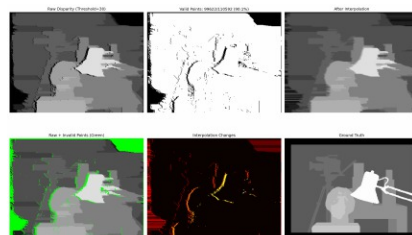
(c) threshold=15



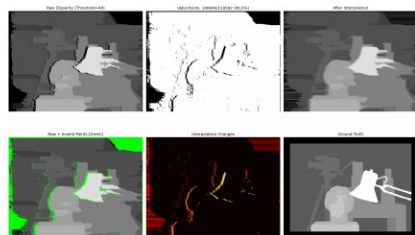
(d) threshold=20



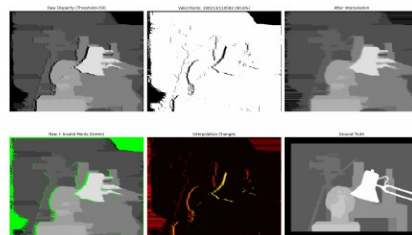
(e) threshold=25



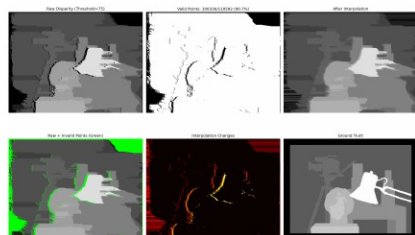
(f) threshold=30



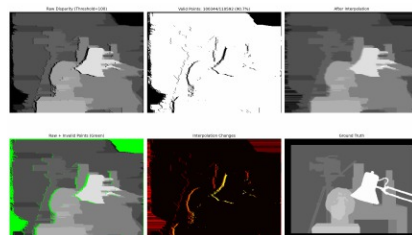
(g) threshold=40



(h) threshold=50



(i) threshold=75



(j) threshold=100

图 19: 不同threshold参数下的效果对比